In [21]:

```
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np


def load_mnist():
    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_dat

    # Normalize the input data - MNIST data is pixel arrays, so divide b
    x_train = x_train/255.0
    x_test = x_test/255.0

    # Output is categorical - map from digit target to vector (e.g. 2 ->
    y_train = keras.utils.to_categorical(y_train, num_classes=10)
    y_test = keras.utils.to_categorical(y_test, num_classes=10)

    return x_train, y_train, x_test, y_test

def build_model(cnn=True):

    model = keras.Sequential()

    # Input is 28x28 image, single channel (grayscale)
    model.add(keras.Input(shape=(28, 28, 1)))

    if not cnn:

        ###  Fully connected neural network ###

        # Input is multidimensional, flattened to single dimension
        model.add(keras.layers.Flatten())
        # Add a hidden layer - units is number of neurons/layer width
        model.add(keras.layers.Dense(units=16, activation="relu", name="
        # TODO add more dense layers and/or vary number of units for inc
        model.add(keras.layers.Dense(units=20, activation="relu", name="
        model.add(keras.layers.Dense(units=12, activation="softmax", nam

    else:

        ###  Convolutional neural network  ###

        # Add convolutional layer - filters is depth of layer output and
        model.add(keras.layers.Conv2D(filters=12, kernel_size=(3, 2), ac
        # Add pooling layer to downscale (MaxPooling downscales by retur
        model.add(keras.layers.MaxPooling2D(pool_size=(2, 2)))
        # TODO add more layers and/or experiment with different number o

        # Flatten internal dimensions before output - additional dense l
        model.add(keras.layers.Flatten())

    # Final model layer - the same for all model architectures
    # Activation is softmax
    model.add(keras.layers.Dense(units=10, activation="softmax", name="o

    return model
```

In [22]:

```
if __name__ == "__main__":
```

```
    # TODO try different values for epochs and learning_rate to improve
    epochs = 25
    learning_rate = 0.4

    x_train, y_train, x_test, y_test = load_mnist()
    model = build_model(cnn=True)  # set cnn=True for convolutional netw

    # Compile model - Stochastic gradient descent is chosen for the opti
    # loss calculation
    optimizer = keras.optimizers.SGD(learning_rate=learning_rate)
    model.compile(optimizer=optimizer, loss="categorical_crossentropy",

    # Show model architecture details and compare parameter counts
    model.summary()

    # Train the model on training data
    history = model.fit(x_train, y_train, epochs=epochs, batch_size=128,

    # Evaluate the model on test data
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=1)
```

**Model: "sequential_5"**

| Layer (type) | Output Shape | P |
|---|---|---|
| conv2d_2 (Conv2D) | (None, 28, 28, 12) | |
| max_pooling2d_2 (MaxPooling2D) | (None, 14, 14, 12) | |
| flatten_5 (Flatten) | (None, 2352) | |
| output (Dense) | (None, 10) | |

**Total params:** 23,614 (92.24 KB)

**Trainable params:** 23,614 (92.24 KB)

**Non-trainable params:** 0 (0.00 B)

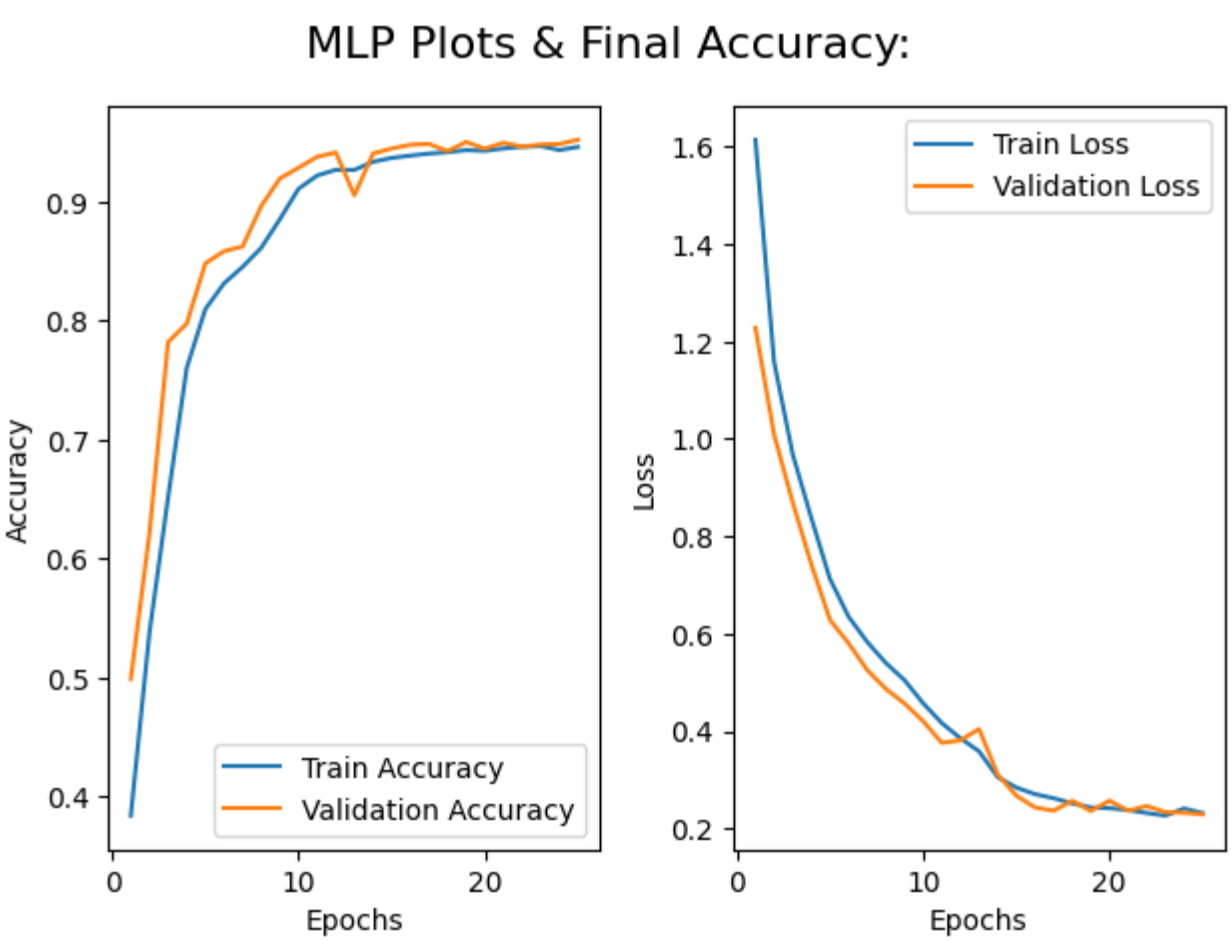
```
Epoch 1/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 14s 31ms/step - accuracy: 0.8170 - loss: 0.61
92 - val_accuracy: 0.9578 - val_loss: 0.1551
Epoch 2/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9535 - loss: 0.15
65 - val_accuracy: 0.9707 - val_loss: 0.1030
Epoch 3/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 27ms/step - accuracy: 0.9674 - loss: 0.11
20 - val_accuracy: 0.9740 - val_loss: 0.0923
Epoch 4/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.9728 - loss: 0.08
99 - val_accuracy: 0.9808 - val_loss: 0.0732
Epoch 5/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 13s 30ms/step - accuracy: 0.9771 - loss: 0.07
78 - val_accuracy: 0.9798 - val_loss: 0.0733
Epoch 6/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 13s 30ms/step - accuracy: 0.9798 - loss: 0.06
76 - val_accuracy: 0.9810 - val_loss: 0.0667
Epoch 7/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 13s 31ms/step - accuracy: 0.9803 - loss: 0.06
43 - val_accuracy: 0.9823 - val_loss: 0.0663
```

```
Epoch 8/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 13s 30ms/step - accuracy: 0.9834 - loss: 0.05
55 - val_accuracy: 0.9810 - val_loss: 0.0674
Epoch 9/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9823 - loss: 0.05
38 - val_accuracy: 0.9827 - val_loss: 0.0664
Epoch 10/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9854 - loss: 0.04
80 - val_accuracy: 0.9815 - val_loss: 0.0758
Epoch 11/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9861 - loss: 0.04
52 - val_accuracy: 0.9840 - val_loss: 0.0617
Epoch 12/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.9864 - loss: 0.04
30 - val_accuracy: 0.9832 - val_loss: 0.0634
Epoch 13/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 13s 30ms/step - accuracy: 0.9871 - loss: 0.04
12 - val_accuracy: 0.9818 - val_loss: 0.0692
Epoch 14/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 13s 30ms/step - accuracy: 0.9874 - loss: 0.03
82 - val_accuracy: 0.9848 - val_loss: 0.0621
Epoch 15/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.9892 - loss: 0.03
57 - val_accuracy: 0.9825 - val_loss: 0.0717
Epoch 16/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.9899 - loss: 0.03
28 - val_accuracy: 0.9835 - val_loss: 0.0721
Epoch 17/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9901 - loss: 0.03
26 - val_accuracy: 0.9820 - val_loss: 0.0723
Epoch 18/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9897 - loss: 0.03
36 - val_accuracy: 0.9822 - val_loss: 0.0691
Epoch 19/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.9912 - loss: 0.02
86 - val_accuracy: 0.9800 - val_loss: 0.0756
Epoch 20/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.9911 - loss: 0.02
92 - val_accuracy: 0.9825 - val_loss: 0.0727
Epoch 21/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.9922 - loss: 0.02
59 - val_accuracy: 0.9830 - val_loss: 0.0706
Epoch 22/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9929 - loss: 0.02
51 - val_accuracy: 0.9773 - val_loss: 0.0832
Epoch 23/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9931 - loss: 0.02
33 - val_accuracy: 0.9825 - val_loss: 0.0734
Epoch 24/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9936 - loss: 0.02
17 - val_accuracy: 0.9812 - val_loss: 0.0772
Epoch 25/25
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 29ms/step - accuracy: 0.9930 - loss: 0.02
33 - val_accuracy: 0.9832 - val_loss: 0.0738
313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 20ms/step - accuracy: 0.9757 - loss: 0.088
1
```

In [ ]:

```python
# -- MLP plots & accuracy --
train_accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']
```

```
train_loss = history.history['loss']
val_loss = history.history['val_loss']

x_values = np.arange(1, len(train_accuracy) + 1)

fig, (ax1, ax2) = plt.subplots(1, 2)
fig.suptitle("MLP Plots & Final Accuracy: ", fontsize=16)
ax1.plot(x_values, train_accuracy, label="Train Accuracy")
ax1.plot(x_values, val_accuracy, label="Validation Accuracy")
ax1.set_xlabel("Epochs")
ax1.set_ylabel("Accuracy")
ax1.legend()

ax2.plot(x_values, train_loss, label="Train Loss")
ax2.plot(x_values, val_loss, label="Validation Loss")
ax2.set_xlabel("Epochs")
ax2.set_ylabel("Loss")
ax2.legend()

fig.tight_layout()
plt.show()

print(f'Accuracy of MLP: {round(test_acc, 3)}, Loss of MLP: {round(test_
# Using: eta=0.4, epochs=25
```

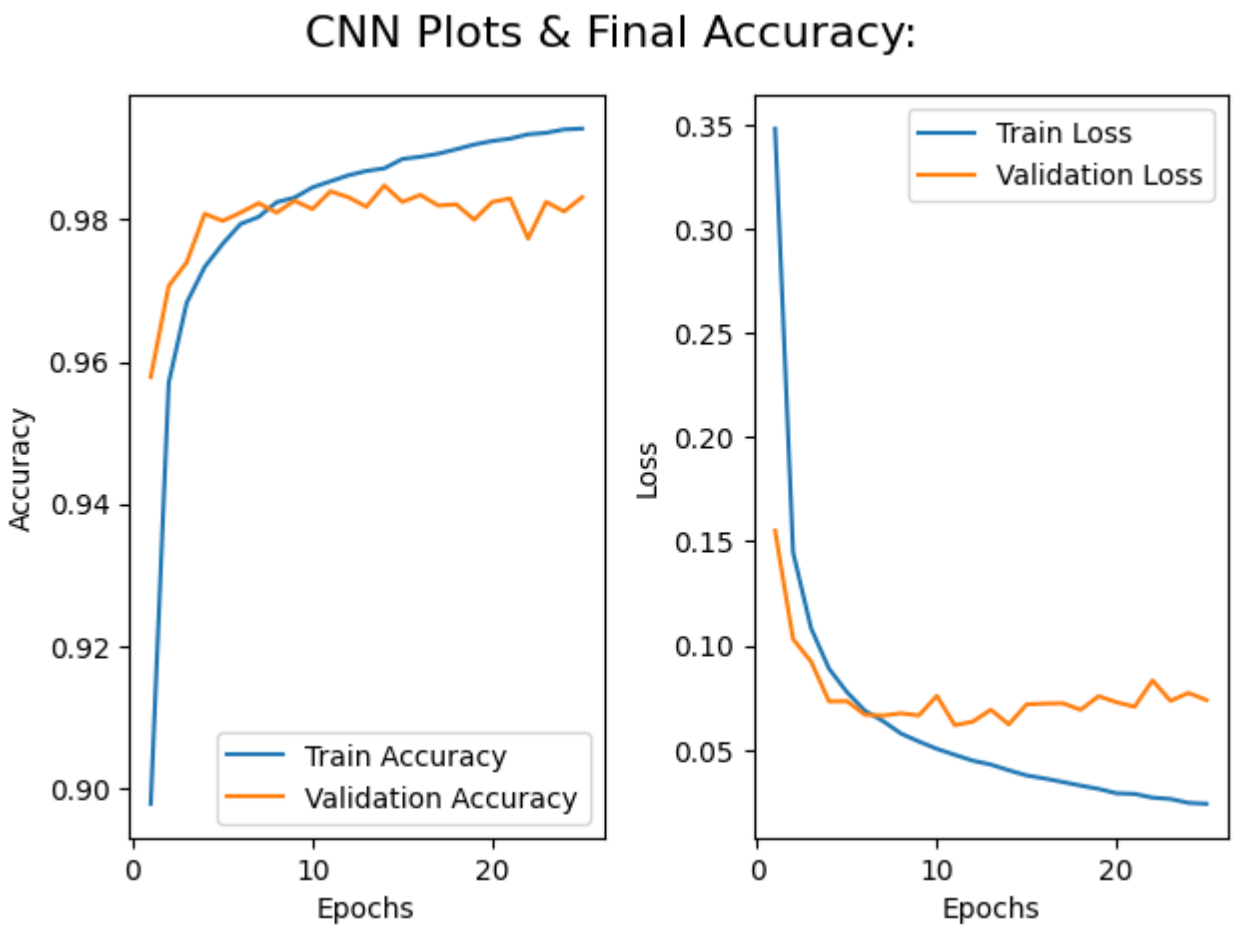



```
Accuracy of MLP: 0.94, Loss of MLP: 0.271
```

In [ ]:

```
# -- CNN plots & accuracy
train_accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']
train_loss = history.history['loss']
val_loss = history.history['val_loss']
```

```
x_values = np.arange(1, len(train_accuracy) + 1)

fig, (ax1, ax2) = plt.subplots(1, 2)
fig.suptitle("CNN Plots & Final Accuracy: ", fontsize=16)
ax1.plot(x_values, train_accuracy, label="Train Accuracy")
ax1.plot(x_values, val_accuracy, label="Validation Accuracy")
ax1.set_xlabel("Epochs")
ax1.set_ylabel("Accuracy")
ax1.legend()

ax2.plot(x_values, train_loss, label="Train Loss")
ax2.plot(x_values, val_loss, label="Validation Loss")
ax2.set_xlabel("Epochs")
ax2.set_ylabel("Loss")
ax2.legend()

fig.tight_layout()
plt.show()

print(f'Accuracy of CNN: {round(test_acc, 3)}, Loss of CNN: {round(test_
# 0.976 og 0.081 m: V. 0.4 og 25 epoker & 8 filter
```




```
Accuracy of CNN: 0.98, Loss of CNN: 0.071
```

Could probably have stopped training earlier, as validation loss seems to have plateaued(danger of overfit).